



Universidad Don Bosco

FACULTAD DE INGENIERÍA

ESCUELA DE COMPUTACIÓN

Desarrollo de aplicaciones con software propietario

Guía 1: Entorno en C#



I. OBJETIVOS

Que el estudiante:

- Reconozca el entorno de herramientas y ventanas que forman al IDE de Visual Studio .Net
- Identifique los tipos de proyectos de C# .NET
- Elabore programas respetando la sintaxis de C#.NET
- Aplique los pasos para la construcción de una aplicación de tipo Windows Form.
- Diferencie uso de los Tipos de Datos y variables que existen en C#.NET
- Declare variables en diferentes ámbitos/alcances

II. INTRODUCCION TEORICA

¿Qué es Visual C#?

C# (pronunciado si sharp en inglés) es un lenguaje de programación orientado a objetos desarrollado y estandarizado por Microsoft como parte de su plataforma .NET, que después fue aprobado como un estándar por la ECMA (ECMA-334) e ISO (ISO/IEC 23270). C# es uno de los lenguajes de programación diseñados para la infraestructura de lenguaje común.

Su sintaxis básica deriva de C/C++ y utiliza el modelo de objetos de la plataforma .NET, similar al de Java, aunque incluye mejoras derivadas de otros lenguajes.

El nombre C Sharp fue inspirado por la notación musical, donde '#' (sostenido, en inglés sharp) indica que la nota (C es la nota do en inglés) es un semitono más alto, sugiriendo que C# es superior a C/C++. Además, el signo '#' se compone de cuatro signos '+' pegados.

Historia y evolución

Durante el desarrollo de la plataforma .NET, las bibliotecas de clases fueron escritas originalmente usando un sistema de código gestionado llamado Simple Managed C (SMC). En enero de 1999, Anders Hejlsberg formó un equipo con la misión de desarrollar un nuevo lenguaje de programación llamado Cool (Lenguaje C orientado a objetos). Este nombre tuvo que ser cambiado debido a problemas de marca, pasando a llamarse C#.3 La biblioteca de clases de la plataforma .NET fue migrada entonces al nuevo lenguaje.

Supone una evolución de Microsoft C y Microsoft C++; es sencillo, moderno, proporciona seguridad de tipos y está orientado a objetos. El código creado mediante C# se compila como código administrado, lo cual significa que se beneficia de los servicios del *Common Language Runtime*.

Estos servicios incluyen interoperabilidad entre lenguajes, recolección de elementos no utilizados, mejora de la seguridad y mayor compatibilidad entre versiones.

C# se presenta como Visual C# en el conjunto de programas Visual Studio .NET. Visual C# utiliza plantillas de proyecto, diseñadores, páginas de propiedades, asistentes de código, un modelo de objetos y otras características del entorno de desarrollo.

.NET Framework es el marco de desarrollo de software para crear y ejecutar aplicaciones en el SO Windows. Sus 2 componentes principales son el Common Language Runtime (CLR) y la biblioteca de clases de .NET Framework.

El CLR es el motor de ejecución que gestiona las aplicaciones en ejecución.

Y la biblioteca de clases proporciona un conjunto de API y tipos para funcionalidades comunes, por ej. incluye API para leer y escribir archivos, conectarse a bases de datos, dibujar y mucho más.

La figura 1 muestra un diagrama con los componentes del IDE de visual studio ya descritos y como interactúan entre si.

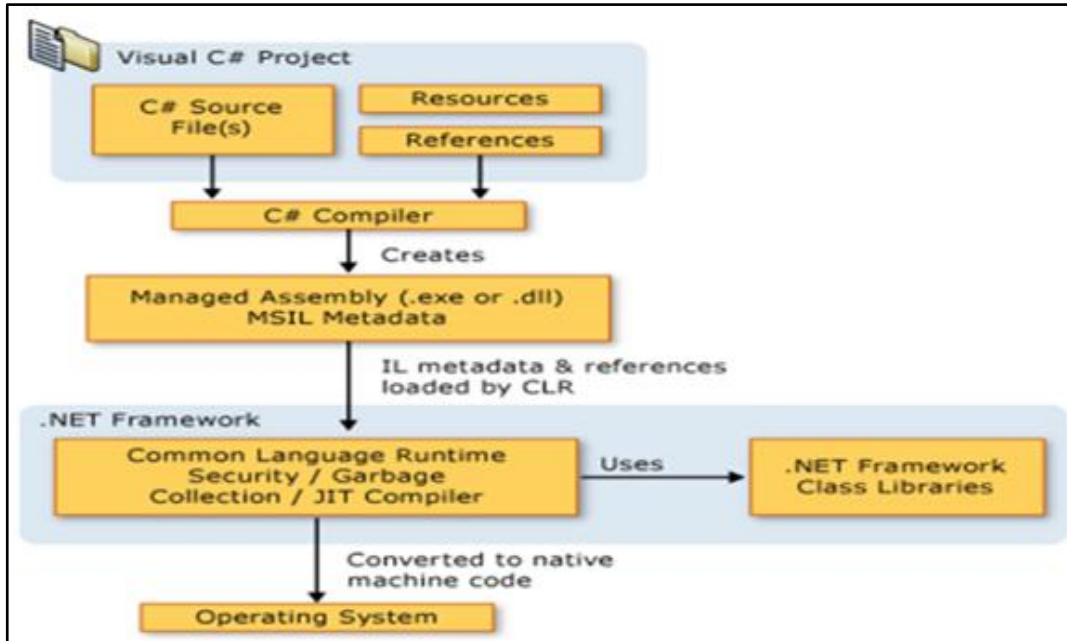


Figura 1: elementos del Framework de Visual Studio en un proyecto con el lenguaje de programación C#

Utilización de C#

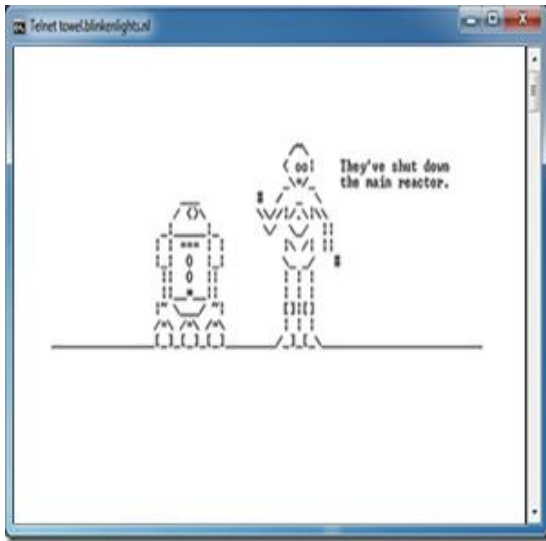
La lista de vistas de App mostradas en la imagen 1 son algunas de las aplicaciones que pueden crearse con C#, por ejemplo:

- Programas de escritorio en Windows XP y versiones posteriores
- Sistemas o sitios web basados en ASPX utilizando C# como lenguaje de programación
- Videojuegos con XNA para pc y XBOX
- Programas en Linux utilizando mono (implementación de CLR multiplataforma)
- Sistemas desktops/web que se necesiten conectar a fuentes de datos

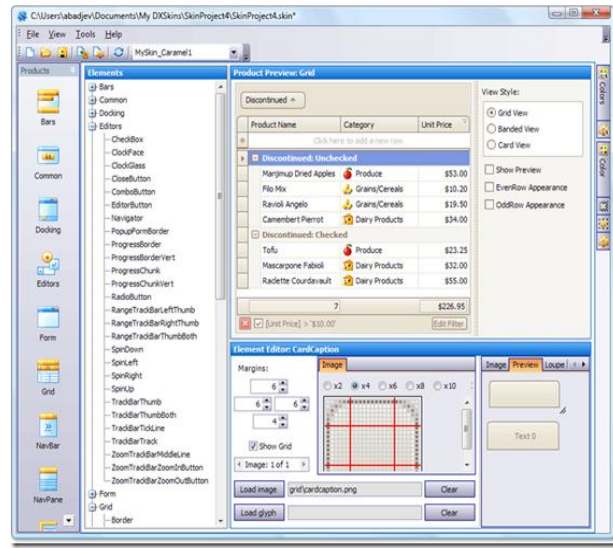
Características

- Deriva de C/C++, y es similar a Java.
- Es Orientado a Objetos.
- Es simple, moderno y de propósito general.
- Tiene revisión estricta de tipos de datos.
- Tiene revisión de límites de vectores.
- Tiene detección de intentos de usar variables no inicializadas.
- Tiene recolección de basura.

1. Aplicación de consola



2. Aplicaciones Tradicionales de cliente Windows



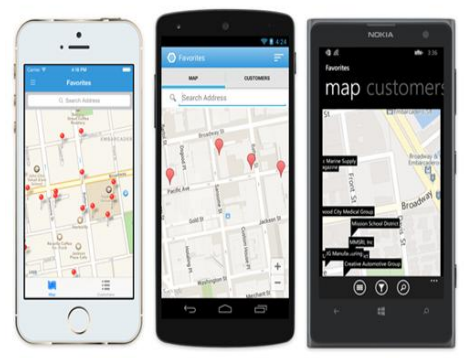
3. Aplicaciones de Windows con Windows Presentation Foundation(WPF)



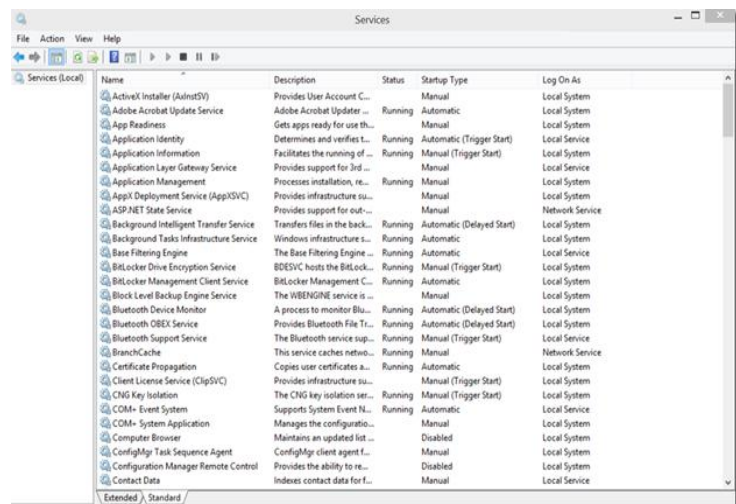
4. Apps Universales para la tienda Windows y Windows Phone



5. Apps Multi-Dispositivo con Xamarin



6. Servicios



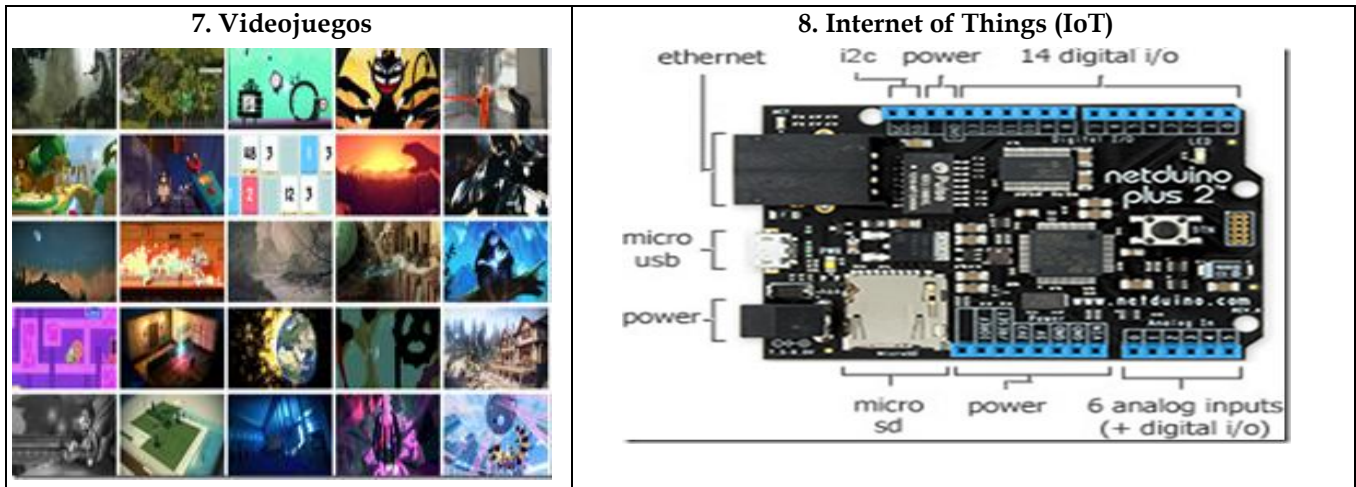


Imagen 1: Algunos tipos de proyectos que pueden desarrollarse con Visual Studio.

Visual C# incluye entre muchas capacidades, la continuación de bucles, la disposición garantizada de recursos, las propiedades de acceso combinado, los tipos de datos sin signo y que aceptan valores NULL, la sobrecarga de operadores, los tipos parciales y genéricos, los eventos personalizados y la comprobación de la compatibilidad con Common Language Specification (CLS).

Ambiente de desarrollo para Aplicaciones Windows

En C# hay diversas de tipos de proyectos disponibles, de los cuales se tratarán a los proyectos de tipo Windows. Estos proyectos presentan un formulario en blanco con las ventanas de apoyo mostradas en la imagen 2.

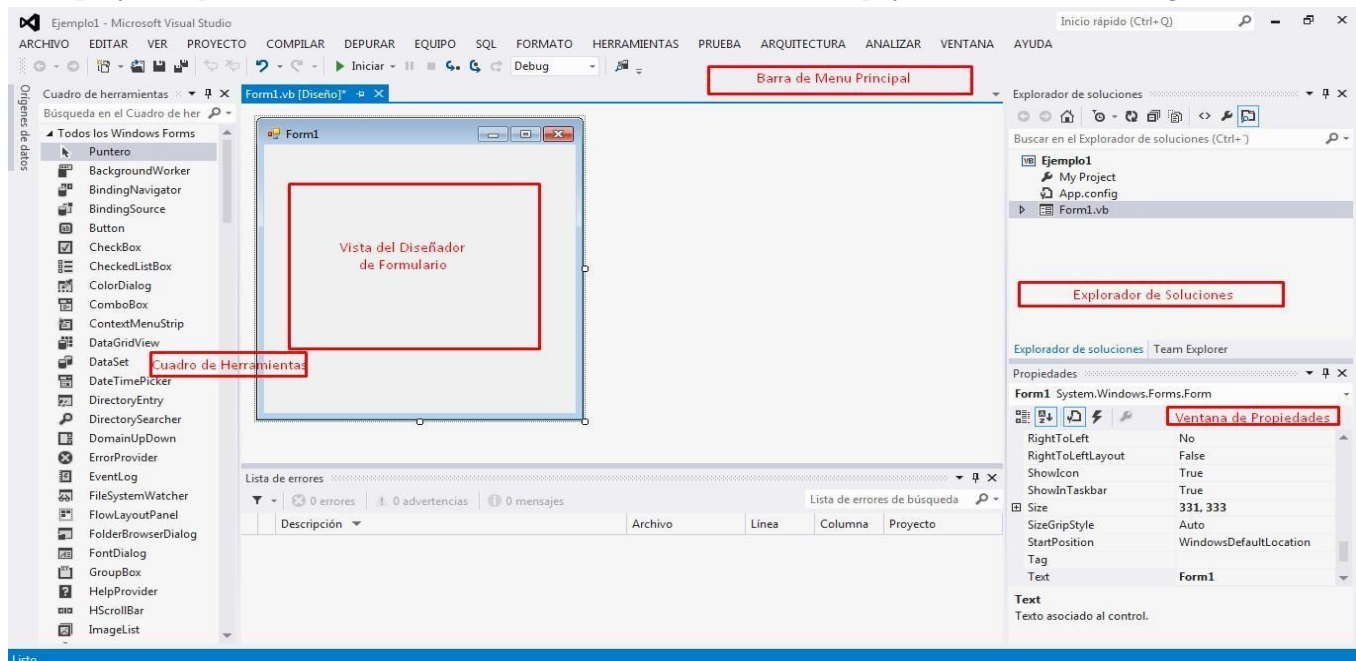


Imagen 2: Vista inicial de un proyecto Windows Forms

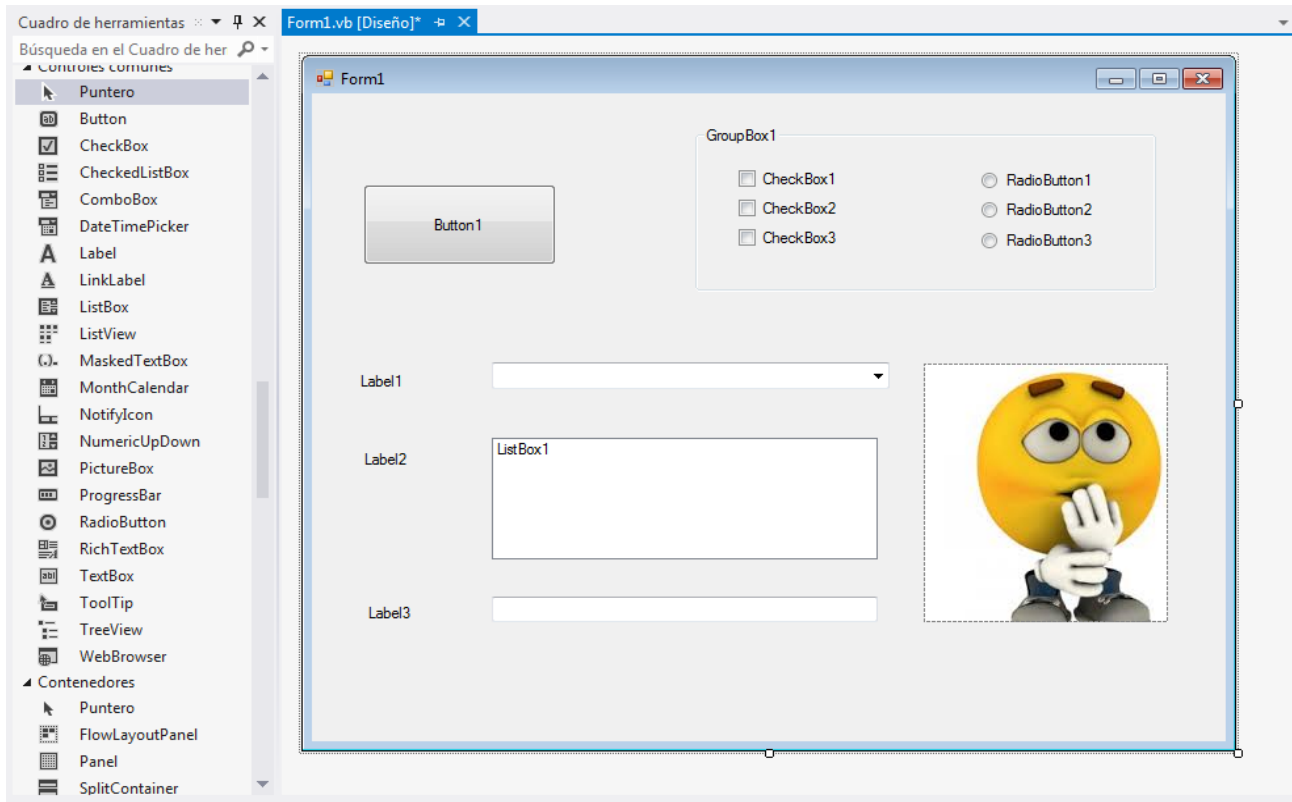


Imagen 3: Vista de los controles más comúnmente utilizados en un formulario

Cuadro de Herramientas

Dentro de un formulario, C# dispone al programador de una serie de controles, los cuales son las maneras de cómo la aplicación puede interactuar con los usuarios a los cuales ira dirigida la aplicación a construir. El .NET Framework consta de una diversidad de controles y objetos para usar en las aplicaciones, clasificados en categorías.

En la imagen 3 se muestran un listado de los controles más comunes utilizados dentro de un form.

Una descripción básica de algunos de estos se muestra en la Tabla 1. Observe que cada control tiene su icono, nombre, prefijo y su descripción.

Icono	Nombre	Prefijo	Descripción
	Button	btn	Se utiliza para iniciar, detener o interrumpir un proceso
	CheckBox	chk	Muestra una casilla de verificación y una etiqueta para texto. Se utiliza en general para establecer opciones.
	ComboBox	cbo	Muestra una lista desplegable de elementos.
	DateTimePicker	dtp	Muestra un calendario gráfico desplegable, que permite que los usuarios seleccionen una fecha o una hora.
	Label	lbl	Muestra texto que los usuarios no pueden modificar directamente.
	ListBox	lst	Muestra una lista de textos, también llamados elementos.
	MaskedTextBox	msk	Captura texto escrito por el usuario, ya sea teniendo un formato de entrada.

Icono	Nombre	Prefijo	Descripción
	PictureBox	pic	Muestra archivos de imágenes, tales como mapas de bits e iconos, gifs, etc.
	RadioButton	rdb	Muestra un botón que puede activarse o desactivarse.
	TextBox	txt	Muestra texto escrito en tiempo de diseño que puede ser editado por los usuarios en tiempo de ejecución.
Components			
	ImageList	img	Es un contenedor de imágenes, para que después se use con otros controles.
	Timer	tmr	Sirve para realizar un conteo de tiempo, en tiempo de ejecución.
Containers			
	FlowLayoutPanel	flp	Es un contenedor de controles que no permite la modificación de la posición del control.
	GroupBox	gpb	Agrupar un conjunto de controles (tales como botones de opción) en un marco con etiqueta.
	Panel	pnl	Agrupar un conjunto de controles en un marco sin etiqueta que permite el desplazamiento.

Tabla 1: Descripción de algunos controles más utilizados de la “barra de controles”

El **prefijo** es una combinación de letras que se le sugiere al programador colocar al inicio del identificador (nombre) del control agregado, de acuerdo con el tipo de este. Con este prefijo, se pretende que, al momento de dar lectura a un código con muchas referencias a objetos utilizados, sea más fácil de entender y comprender.

Estructura de un programa en C#

C# es bastante sencillo en cuanto a su sintaxis. Alguien con experiencia en programación con lenguaje C/C++, Java o Perl, no debería tener ningún problema de adaptación. Sin embargo, posee cierto tipo de dificultades que se superan en la medida se profundiza en la programación de este.

A continuación, mostraremos los tópicos relacionados a la sintaxis de C#.

Clase estándar

Un módulo agrupa código fuente de una aplicación y se almacena en un archivo .cs. Para definir un módulo se usa la sintaxis siguiente:

```
namespace Nombre_Aplicacion
{
    public partial class Nombre_Formulario : Form
    {
        public Form1()
        {
            [SENTENCIAS]
        }
    }
}
```

Procedimientos

Un procedimiento es un sub-programa o rutina al que se le asigna un cierto nombre y que realiza unas determinadas operaciones. La ventaja que tiene utilizar procedimientos es que su código se escribe una sola vez pero pueden ser ejecutados desde tantos puntos del script como se desee.

Un procedimiento puede recibir una serie de parámetros que variarán su funcionamiento. Los parámetros a recibir (de qué tipo es cada uno y para qué se utiliza) se determinan al escribir el procedimiento.

Dentro del código que determina el funcionamiento de un procedimiento se pueden a su vez ejecutar otros procedimientos, incluso sí mismo de forma recursiva.

Funciones:

Una función es un procedimiento que, adicionalmente, devuelve un valor como resultado de su ejecución. La ventaja de las funciones es que pueden utilizarse en asignaciones, expresiones matemáticas, etc.

Sintaxis:

Declaración y programación (escritura) de un procedimiento:

```
public int Maximo (int x, int y) {

    int mayor;

    if (x==y) return -1;

    else

        if (x>y) return x;

        else return y;

}
```

En la primera parte de la declaración:

```
public int Maximo (int x, int y)
```

En donde **public**, es el nivel de acceso, indica que puede ser accedida desde cualquier scrip, dentro o fuera del archivo donde se declaró.

Luego, **int** es el valor que retornará la función, en este caso como es un numero el que tiene que retornar, pero no siempre es así, algunas veces, hay funciones que piden de argumento un string y retornan un bool u otro tipo de datos.

Maximo, es el nombre de la función

(**int x, int y**) son los argumentos de la función, los valores que necesita para operar, en este caso son dos, siempre se separan con una coma y se debe de definir el nombre y el tipo de datos.

Y dentro de esta función utiliza los argumentos que pide y realiza la comparación, lo de adentro es un simple if...

Un evento es un mensaje que envía un objeto cuando ocurre una acción. La acción podría ser causada por la interacción del usuario, como un clic del botón, o podría ser iniciado por otra lógica de programa, como cambiar un valor de propiedad. El objeto que provoca el evento se conoce como emisor del evento. El emisor del evento no sabe qué objeto o método recibirá (controlará) los eventos que genera.

El evento normalmente es un miembro del emisor del evento; por ejemplo, el evento Click es un miembro de la clase Button, y el evento PropertyChanged es un miembro de la clase que implementa la interfaz INotifyPropertyChanged.

Para definir un evento, se utiliza la palabra clave **event** (en C#) o **Event** (en Visual Basic) en la signatura de la clase de eventos y se especifica el tipo de delegado para el evento. Los delegados se describen en la sección siguiente.

Normalmente, para generar un evento, se agrega un método marcado como **protected** y **virtual** (en C#) o **Protected** y **Overridable** (en Visual Basic). Asigne a este método el nombre **OnEventName**; por ejemplo, **OnDataReceived**. El método debe tomar un parámetro que especifica un objeto de datos de evento. Este método se proporciona para permitir que las clases derivadas reemplacen la lógica para generar el evento. Una clase derivada siempre debería llamar al método **OnEventName** de la clase base para asegurarse de que los delegados registrados reciben el evento.

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("HOLA MUNDO");
}
```

Manejo de Variables en C#.Net

Puntuadores de sentencias.

Los puntuadores son caracteres especiales que delimitan bloques de código, y el fin de una sentencia.

```
{ línea_de_codigo1; línea_de_codigo2, línea_de_codigon; ... }
```

Las llaves agrupan múltiples sentencias dentro de un bloque de sentencias. Las sentencias pueden ocupar una o varias líneas. Esto último resulta útil para asuntos de legibilidad del código fuente. El terminador o delimitador de sentencias en C# es el punto y coma (;), el mismo que se utiliza en C/C++. Por lo tanto, cuando desee terminar una sentencia para iniciar otra debe utilizar el punto y coma.

Comentarios.

C# utiliza los estilos de comentarios del lenguaje C/C++ y del shell de la interfaz de comandos de Unix y Linux. Estos son el comentario de una sola línea `"/"` y el comentario de bloque `"/ * ... */"`, también utilizados en el lenguaje C.

Veamos algunos ejemplos:

1. Comentarios de línea simple: Por ejemplo

```
int x=3;//Asigna 3 a x, es útil para documentar una línea simple de código.
```

2. Comentarios de múltiples líneas:

```
int x = 12 * 30; /* Este comentario
se expande en dos líneas */
```

Tipos de datos disponibles

C# contiene dos categorías generales de tipos de datos integrados: tipos de valor y tipos de referencia. El término tipo de valor indica que esos tipos contienen directamente sus valores.

La lista de tipos de datos para manejar valores enteros se muestra en la tabla 2.

Y, por otra parte, los tipos de *coma flotante* (listados en tabla 3) pueden representar números con componentes fraccionarios. Existen dos clases de tipos de coma flotante: float y double.

El tipo double es el más utilizado porque muchas funciones matemáticas de la biblioteca de clases de C# usan valores double.

Quizá, el tipo de coma flotante más interesante de C# es decimal, dirigido al uso de cálculos monetarios. La aritmética de coma flotante normal está sujeta a una variedad de errores de redondeo cuando se aplica a valores decimales.

El tipo decimal elimina estos errores y puede representar hasta 28 lugares decimales.

Tipo de datos de enteros				
Tipo	Equivalente BCL	Tamaño	Rango	Significado
byte	System.Byte	8-bit (1-byte)	0 a 255	Entero sin signo
sbyte	System.SByte	8-bit (1-byte)	-128 a 127	Entero con signo
short	System.Int16	16-bit (2-byte)	-32.768 a 32.767	Entero corto con signo
ushort	System.UInt16	16-bit (2-byte)	0 a 65.535	Entero corto sin signo
int	System.Int32	32-bit (4-byte)	-2.147.483.648 a 2.147.483.647	Entero medio con signo
uint	System.UInt32	32-bit (4-byte)	0 a 4.294.967.295	Entero medio sin signo
long	System.Int64	64-bit (8-byte)	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807	Entero largo con signo
ulong	System.UInt64	64-bit (8-byte)	0 a 18.446.744.073.709.551.615	Entero largo sin signo

Tabla 2: Lista de tipos de datos básicos y su rango de valores aceptados

Tipo de datos de coma flotante				
Tipo	Equivalente BCL	Tamaño	Rango	Significado
float	System.Single	32-bit (4-byte)	±1.401298E-45 a ±3.402823E+38	Coma flotante corto
double	System.Double	64-bit (8-byte)	±4.94065645841246E-324 a ±1.79769313486232E+308	Coma flotante largo
decimal	System.Decimal	128-bit (16-byte)	-7.9228162514264337593543950335 a +7.9228162514264337593543950335	Coma flotante monetario

Tabla 3: Lista de tipos de datos para números con punto decimal y su rango de valores aceptados

Los caracteres en C# no tienen un tamaño de 8 bits como en otros muchos lenguajes de programación, sino que usa un tamaño de 16 bits llamado Unicode al cual se le llama *char*. Se manejan con el tipo de dato *char* mostrado en la tabla 4. No existen conversiones automáticas de tipo entero a *char*.

Tipo de datos de caracteres				
Tipo	Equivalente BCL	Tamaño	Rango	Significado
char	System.Char	16-bit (2-byte)	'\u0000' a '\uFFFF'	Carácter unicode

Tabla 4: Lista de tipos de datos para números con punto decimal y su rango de valores aceptados

Para los tipos de datos lógicos, llamados también booleanos, existe el tipo de dato *bool* (descrito en la Tabla 5). No existen conversiones automáticas de tipo entero a *bool*.

Tipo de datos lógicos				
Tipo	Equivalente BCL	Tamaño	Rango	Significado
bool	System.Boolean	8-bit (1-byte)	true o false	Verdadero o falso

Tabla 5: tipo de dato bool y su rango de valores disponibles.

Declaración de Variables

Las variables son *identificadores* asociados a valores. Se declaran indicando el tipo de dato que almacenará y su identificador. *Una variable representa un valor numérico o de cadena o un objeto de una clase.* El valor que la variable almacena puede cambiar, pero el nombre sigue siendo el mismo. Una variable es un tipo de campo. El código siguiente es un ejemplo sencillo de cómo declarar una variable de entero, asignarle un valor y, a continuación, asignarle un nuevo valor.

```
<tipo_de_dato> nombre_variable [= valor];
int x = 1; // x almacena el valor de 1
x = 2;    // ahora, x almacena el valor de 2
```

En C#, las variables se declaran con un tipo de datos y una etiqueta concretos como se ha visto en el ejemplo anterior. Si hasta ahora sólo ha utilizado lenguajes con tipos definidos de forma imprecisa como JavaScript, estará acostumbrado a emplear el mismo tipo "var" para todas las variables, pero en C# se tiene que especificar si la variable es de tipo int, float, byte, short u otro cualquiera entre más de 20 tipos de datos diferentes. El tipo especifica, entre otras cosas, *la cantidad de memoria exacta* que se debe asignar para almacenar el valor cuando la aplicación se ejecuta.

Un identificador puede:

- Empezar por "_".
- Contener caracteres Unicode en mayúsculas y minúsculas (sensible a mayúsculas y minúsculas).

Un identificador no puede:

- Empezar por un número.
- Empezar por un símbolo, ni aunque sea una palabra clave.
- Contener más de 511 caracteres.

Constantes

Las constantes son valores inmutables, y por tanto no se pueden cambiar. Cuando se declara una constante con la palabra clave *const*, también se debe asignar el valor. Tras esto, la constante queda bloqueada y no se puede cambiar su valor. Son implícitamente estáticas (*static*).

```
const double PI = 3.1415;
```

Otra forma alternativa para la manipulación de constantes dentro de C# es haciendo uso de miembros de solo lectura, que a diferencia de *const*, no requiere que se asigne el valor al mismo tiempo que se declara. Pueden ser miembros de la instancia o miembros estáticos de la clase (static).

```
readonly double E;
E = 2.71828;
```

Tratamiento de cadenas

- El tipo de dato para las cadenas de caracteres es string.
- Realmente la palabra clave *string* es un alias de la clase System.String de la plataforma .NET.

- En C# las cadenas son objetos y no una matriz de caracteres; aun así, se puede obtener un carácter arbitrario de una cadena por medio de su índice (pero no modificarlo).
- Las cadenas son inmutables, una vez creadas no se pueden modificar, solo se pueden copiar total o parcialmente.
- El operador == determina si dos referencias hacen referencia al mismo objeto, pero al usar dicho operador con dos variables tipo string se prueba la igualdad del contenido de las cadenas y no su referencia. Sin embargo, con el resto de los operadores relacionales, como < y >=, sí se comparan las referencias.
- Se pueden concatenar (unir) dos cadenas mediante el operador +.
- Las cadenas se pueden usar en las instrucciones switch.

1. Declarar una cadena de caracteres (como si fuera una variable de un tipo de dato como int o double):

```
string texto = "Cadena de caracteres";
string texto = new System.String("Cadena de caracteres"); // Equivalente al anterior
```

2. Longitud de una cadena:

```
string texto = "Cadena de caracteres";
int i = texto.Length; // Retornará '20'
```

3. Comparar dos cadenas:

```
bool b = "texto" == "texto"; // Retornará 'true', ya que ambas cadenas contienen "texto"
```

4. Concatenar cadenas:

```
string texto = "Cadena de" + " caracteres"; // Nótese el espacio antes de "caracteres", si no se pusiera, aparecería junto: "decaracteres"
```

Códigos o secuencias de escape

En C#, al igual que en otros lenguajes existen ciertos caracteres que generan problemas cuando se encuentran dentro de cadenas, debido a que dentro del lenguaje se interpretan de alguna forma especial. Para poder visualizar correctamente dichos caracteres en una operación de salida se utilizan secuencias de escape que involucran dichos caracteres especiales.

La tabla 6 muestra varios de estos caracteres especiales con su correspondiente secuencia de escape:

Secuencia de escape	Significado
\b	Espacio hacia atrás (<i>backspace</i>)
\a	Alerta (timbre)
\f	Cambio de página (<i>form feed</i>)
\n	Cambio de línea (<i>line feed</i>)
\r	Retorno de carro (<i>carriage return</i>)
\t	Tabulación horizontal
\v	Tabulación vertical
\\	Barra inversa (<i>backslash</i>)
\'	Comilla simple
\"	Comilla doble
\0	Valor nulo

Tabla 6: lista de algunos de los caracteres de escape mas usados

Expresiones en C#: Prioridad en los Operadores

Los operadores son símbolos especiales que se utilizan en los lenguajes de programación para poder realizar operaciones con las expresiones. Como lo que se obtiene en dichas operaciones es un valor, el resultado también será una expresión.

Los operadores se pueden clasificar como: unarios (que operan sobre un único valor o expresión), binarios (que operan sobre dos valores o expresiones) y ternarios (que consta de tres valores o expresiones)

C# proporciona un amplio conjunto de operadores (ver la Tabla 7), que son símbolos que especifican las operaciones que se deben realizar en una expresión. Operaciones con tipos enteros como ==, !=, <, >, <=, >=, binary +, binary -, ^, &, |, ~, ++, -- y sizeof() son generalmente permitidas en enumeraciones. Además, el usuario puede sobrecargar muchos de los operadores, es decir, cambiar su significado al aplicarlos a un tipo definido por el usuario.

En la tabla siguiente se muestran los operadores de C# agrupados por orden de prioridad. Los operadores dentro de cada de grupo tienen la misma prioridad.

Categorías	Operadores	Uso
Primario	x.y	Acceso a miembros, sea propiedad o método
	f(x)	Orden de operaciones en una expresión, especificar conversión de tipos
	a[x]	Utilizados en matrices, indizadores y atributos
	x++	Incremento postfijo en 1, devuelve el valor de x
	x--	Decremento postfijo en 1, devuelve el valor de x
	new	Creación de nuevos objetos e invocación de constructores
	typeof	Obtiene el tipo de dato asociado
	checked	Habilitar explícitamente la comprobación de desbordamiento para operaciones aritméticas y conversiones de tipo integral.
	unchecked	Suprimir la comprobación de desbordamiento para operaciones aritméticas y conversiones de tipo integral.
	sizeof	Se usa para obtener el tamaño en bytes de un tipo no administrado
Unario	->	El operador -> combina la des referenciación de un puntero y el acceso a un miembro.
	+x	Suma de operandos y en el caso de cadenas su concatenación
	-x	Resta de operandos
	!x	Negación del operando utilizado con valores de tipo bool
	++x	Incremento prefijo en 1, devuelve el valor de x+1
	--x	Decremento prefijo en 1, devuelve el valor de x-1
	&x	Devuelve la dirección de memoria de su operando
Multiplicativo	*x	Operador de desreferenciación que permite leer y escribir en un puntero
	x * y	Multiplicación de operandos
	x / y	División de operandos
Sumatorio	x % y	Residuo de dividir x entre y
	x + y	Suma de operandos
	x - y	Resta de operandos
Desplazamiento	x << y	Desplaza x a la izquierda y cantidad de bits
	x >> y	Desplaza x a la derecha y cantidad de bits

Comprobación de tipos y relacionales	$x < y$	Devuelve TRUE si x es menor que y , FALSE caso contrario
	$x > y$	Devuelve TRUE si x es mayor que y , FALSE caso contrario
	$x \leq y$	Devuelve TRUE si x es menor o igual que y , FALSE caso contrario
	$x \geq y$	Devuelve TRUE si x es mayor o igual que y , FALSE caso contrario
	<code>is</code>	Devuelve TRUE si el objeto es compatible con cierto tipo
Igualdad	$x == y$	Devuelve TRUE si x es igual que y , FALSE caso contrario
	$x != y$	Devuelve TRUE si x es diferente que y , FALSE caso contrario
AND condicional	$x \&\& y$	Devuelve TRUE si x es TRUE y y es TRUE, FALSE caso contrario
OR condicional	$x y$	Devuelve TRUE si x es TRUE o y es TRUE, FALSE caso contrario
Condicional	<code>?:</code>	Devuelve uno de los valores dependiendo del resultado de la expresión booleana: <code>condition ? first_expression_true : second_expression_false;</code>
Expresión de asignación y lambda	$x = y$	Asigna a x el valor de y
	$x += y$	Actualiza el valor de x con la suma de x con y
	$x -= y$	Actualiza el valor de x con la resta de x con y
	$x *= y$	Actualiza el valor de x con el producto de x por y
	$x /= y$	Actualiza el valor de x con la división de x por y
	$x \%= y$	Actualiza el valor de x con el residuo o módulo de x entre y

- Para mayor referencia sobre otros operadores y operadores adicionales visite:
- <http://msdn.microsoft.com/es-sv/library/6a71f45d.aspx>

Tabla 7: lista de algunos de los caracteres de escape mas usados

FUNCIONES DE CONVERSION DE TIPOS

Supongamos que dejamos que un usuario introduzca un valor numérico por pantalla y queremos realizar cálculos con ese valor. Por lo tanto, el valor introducido por el usuario será del tipo `System.String` para poder realizar conversiones en visual C# podemos convertirlo de dos formas, por ejemplo si queremos pasarlo a `System.Int32`.

```
//sintaxis: Convert.Tipo_dato_a_convertir(variable_a_convertir);
int valorA = Convert.ToInt32(valorUsuario);
//sintaxis: Tipo_dato_a_convertir.Parse(Valor_a_convertir);
int valorB = Int32.Parse(valorUsuario);
```

Cuando utilizamos la segunda opción, el método `Parse()`, si el usuario no ha introducido ningún valor (`null`) recibiremos una excepción del tipo `System.FormatException`. Lo que indica que el formato del argumento no cumple las especificaciones del parámetro del método invocado.

En cambio, cuando utilizamos la primera opción, los métodos de la clase `System.Convert`, si el valor pasado al método es `null`, el método `ConovertToInt32()` devolverá un valor numérico, devolverá cero en todos los casos que el valor de entrada sea `null`.

SENTENCIAS DE CONTROL CONDICIONAL

En C# se dispone de las siguientes estructuras de tomas de decisiones:

if [else]

switch

Ambas estructuras son equivalentes en su ejecución a las estructuras if-else y switch de lenguaje C, respectivamente. En la tabla 8 se muestra la sintaxis de uso de este par de sentencias de control.

Estructura if [else]	Estructura switch
<pre> if (i == 2) { // ... } else if (i == 3) { // ... } else { // ... } </pre>	<pre> switch (i) { case 1: ... break; case 2: case 3: ... break; default: ... break; } </pre>

Tabla 8: sintaxis generales de estructuras condicionales de C#

III. MATERIALES Y EQUIPO

Para la realización de la guía de práctica se requerirá lo siguiente:

No.	Requerimiento	Cantidad
1	Guía de Laboratorio #1	1
2	PC con Microsoft Visual Studio 2019 (o posterior) y .Net Framework 4.7 (o posterior)	1
3	Memoria USB	1

IV. PROCEDIMIENTO

Para esta práctica, ubique a la carpeta del Escritorio o Documentos del S.O. de su PC y luego, procede a crear una carpeta con el nombre de **DSPpractica1**, en la cual va a guardar todos los archivos de esta práctica y de análisis de resultados.

PARTE 1: Creando una Aplicación Windows

Pasos para la creación de aplicaciones en Visual C#

1. Abre la aplicación **Visual Studio Community 2022**. En la pantalla de inicio (ver Imagen 4), haz clic en “**Crear un proyecto**”.
2. Luego, tal como se muestra en la imagen 5, en la ventana “**Crear un nuevo proyecto**”, utiliza los siguientes filtros o búsqueda para ubicar la plantilla del tipo de proyecto a utilizar:
 - Lenguaje: **C#**
 - Plataforma: **Windows**
 - Tipo de proyecto: **Escritorio**

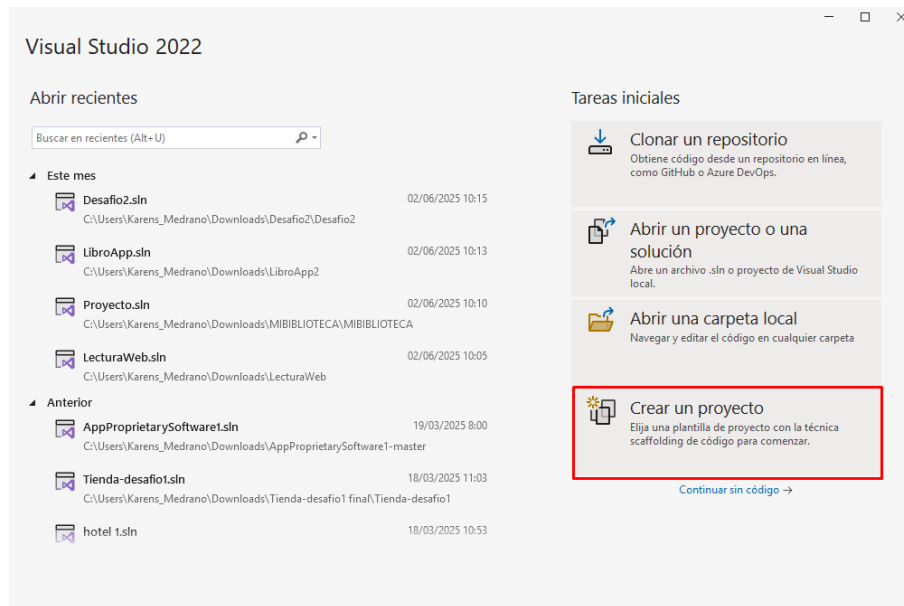


Imagen 4: pantalla inicial de Visual Studio.

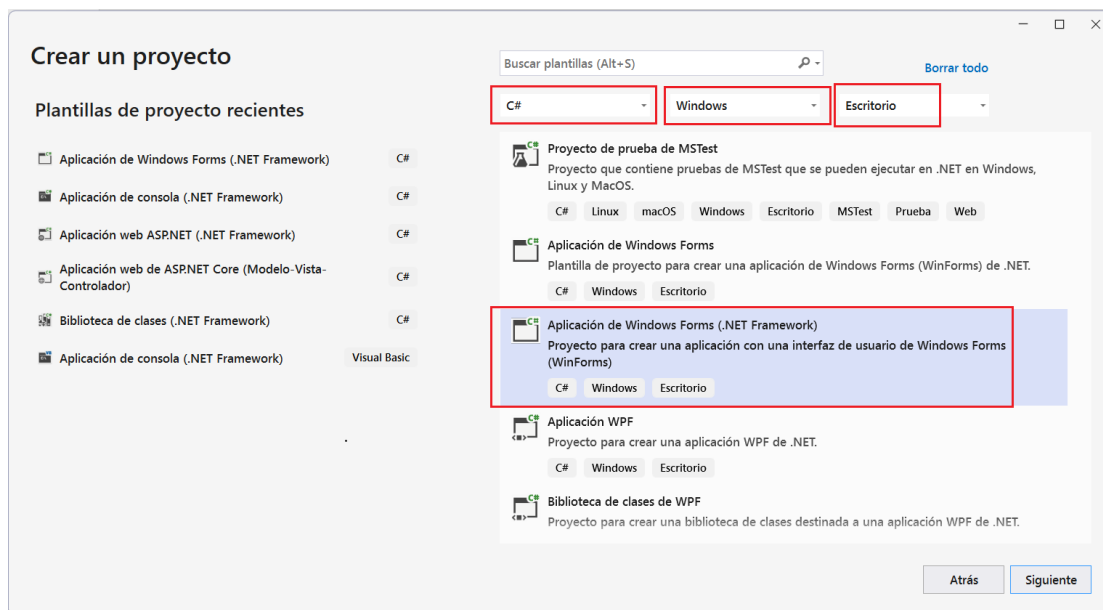


Imagen 5: Selección de filtros para ubicar a plantilla del proyecto Windows Form.

3. Selecciona la plantilla **Aplicación Windows Forms (.NET Framework)** y haz clic en boton “**Siguiente**”.
4. Como se muestra en la imagen 6, para configurar el nuevo proyecto, asigne como *Nombre del proyecto* el nombre **Ejemplo1**.

En la opción *Ubicación*, asegure de abrir a la carpeta solicitada al inicio de este procedimiento.

Marque la opción *Colocar la solución y el proyecto en el mismo directorio*.

Para la elección del Framework, seleccione la versión .NET Framework 4.8 o una superior.

5. Haga clic en el boton **Crear**.

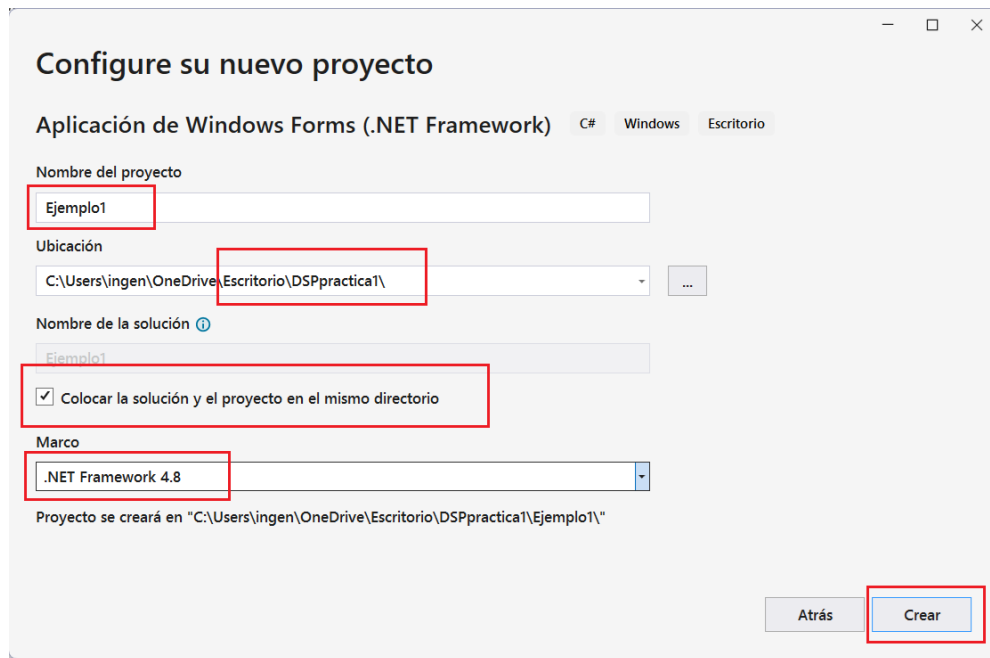


Imagen 6: Nombre del proyecto y su ubicación en la unidad de almacenamiento de su PC

6. Ahora proceda a incluir en el area de Diseño del Form, a c/u de los controles mostrados a la izquierda de la imagen 7.

Localice el panel *Cuadro de herramientas* y determine el tipo control a utilizar.

Hay 2 maneras de agregar los controles deseados al formulario:

- A. Ubica y hace clic sobre el Control elegido de la barra herramientas y luego, hacer clic en el area del form que usara ese control.
- B. Arrastrar con el ratón al Control elegido de la barra herramientas y soltarlo dentro del area del form.

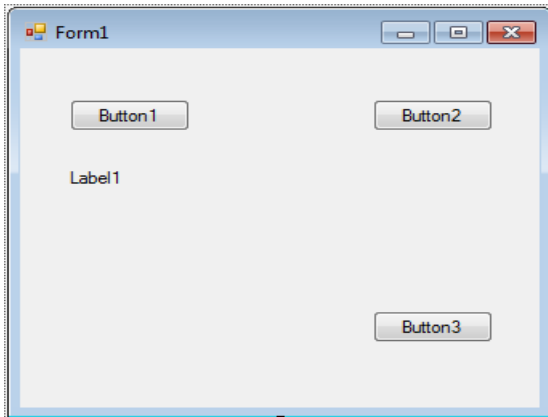
Establecer las propiedades de los Controles

Las propiedades de un Control, cambian la apariencia y/o el funcionamiento de este ante el usuario que usa el form.

Una propiedad en Visual C# representa un atributo de un objeto, en este caso, un Control. Por ejemplo:

- ✓ Uno de los atributos de un control Button es el texto que esta muestra. En este caso, se puede modificar al asignar la propiedad **Text**. La mayoría de éstas modificaciones también se pueden establecer o modificar dentro del código del programa
- ✓ Las propiedades pueden tomar muchos tipos diferentes de valores además del texto.
- ✓ Si se cambia el tamaño de un control o se reubica, también se actualizan las propiedades **Size** y **Location** que determinan el tamaño y la ubicación de un control en el formulario.

7. Continuando con el procedimiento, en la tabla de la imagen 7 se detalla a los diferentes controles colocados en el Form1 y cada propiedad (con su nuevo valor) que deberá modificar antes de continuar. Observe uso de prefijos en el identificador de c/control (que es definido en propiedad Name de c/control).



CONTROL	PROPIEDAD	Nuevo VALOR
Form1	Text	Pruebas de ámbito de variables
	BackColor	Un color Personalizado a su gusto
Button1	Name	btnContar
	Text	Contar
Button2	Name	btnReinicio
	Text	Reiniciar conteo
Button3	Name	btnFin
	Text	Salir del programa

Imagen 7: Vista Formulario del Ejemplo 1 y los controles requeridos (con propiedades a modificar).

Agregar el código y la funcionalidad al Programa

8. Para agregar código a los diferentes controles, es necesario cambiarse al editor de código.

Para ello, haga clic secundario sobre el área del form (o sobre cualquiera de sus controles incluidos en este) y del menú contextual, seleccione *Ver código*.

Se abrirá una ficha (ventana) denominada Editor de código, como lo muestra la imagen 8.

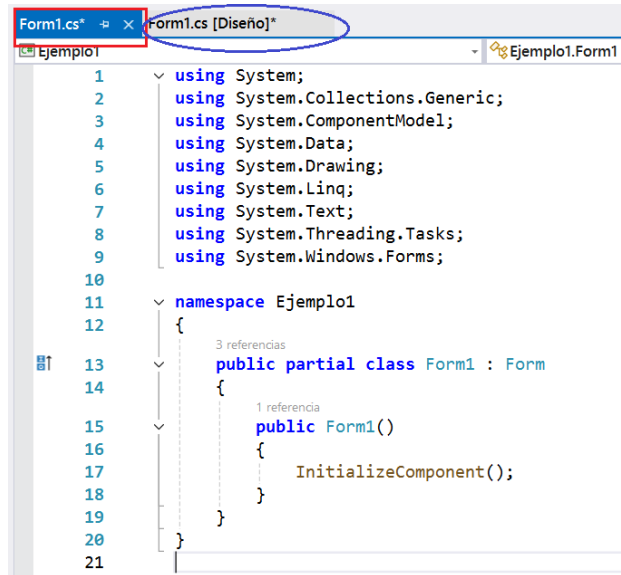


Imagen 8: Vista del editor de código para editar la funcionalidad de la aplicación

9. Retorne nuevamente a la vista de diseño del form, haciendo clic en la ficha que tiene el mismo nombre del Form1.css: Form1.css [Diseño]
10. Retorne a la vista de código, pero esta vez presione solamente la tecla F7. Esta es otra forma de cambiarse a la ficha de edición de código.

11. Observe nuevamente al código de la imagen 8:

- Primero, se listan las referencias de los namespaces del Net.Framework sugeridas por Visual Studio para su proyecto Windows Forms
- Luego, la apertura de un namespace propio (cuyo nombre coincide con el de su proyecto).
- Finalmente, se define una clase (llamada igual que el nombre del form Form1) para gestionar la funcionalidad del formulario para programar a los objetos insertados en el mismo.
- Dentro de la clase, se define una función, cuyo nombre coincide con el de la clase Form1. A esta función se le llama Metodo Constructor.

12. Retorne a la vista de diseño y haga doble clic sobre el botón **btnContar**. Observe que se retorna a la vista de código y también, se ha incluido una nueva función, así:

```
1 referencia
private void btnContar_Click(object sender, EventArgs e)
{
}
}
```

A esta función se le llama un *Controlador de evento*, también denominado **procedimiento manejado por evento**. Cualquier código dentro de esta función se ejecuta cada vez que sucede el evento programado para ese control.

En este ejemplo, esta función se ejecutara solamente cuando usuario haga clic sobre el botón **btnContar**.

13. Ejecute el programa creado hasta aquí. Lo puede hacer de tres formas diferentes:

- Presionando la tecla F5
- Eligiendo en el menú Depurar -> Iniciar Depuración
- Haciendo clic en la Barra Estándar en el 1er icono del siguiente trio  de la barra de botones Estándar.

14. Finalice la ejecución del programa en diseño, seleccionando el 3er boton del trio de botones mencionado en paso anterior.

PARTE 2: Ambito/Alcance de uso de una variable

El **ámbito** o **alcance** (*scope*) determina la parte del código donde una variable puede ser utilizada.

Se define principalmente por los bloques de código delimitados por llaves { }. Fuera de su ámbito, una variable, por ej., llamada **x** no existe y C# marcará un error de "El nombre **"x"** no existe en el contexto actual".

15. En esta practica inicial, nos enfocaremos en la redacción de código dentro del ámbito de una clase. En la imagen a continuación, se les resalta los ámbitos existentes de la clase Form1:

```
3 referencias
public partial class Form1 : Form           ambito: Clase
{
    |
    1 referencia
    public Form1()
    {
        InitializeComponent();             ambito:
                                            funcion constructor
    }

    1 referencia
    private void btnContar_Click(object sender, EventArgs e)
    {
                                            ambito:
                                            funcion btnContar_Click
    }
}
```

16. Desplace el cursor justo después de la llave de apertura de la clase Form1 y presione tecla Enter.
En este momento se encuentra en el ámbito de la clase Form1. Las variables declaradas aquí serán reconocidas por todos los miembros (funciones) definidos dentro de la clase Form1.
17. Declare a 2 variables llamadas **mensa** (declárela del tipo *string*) y a **conta** (de tipo *int*), respectivamente.
18. Retorne a la vista de diseño del form y haga doble clic en cualquier parte vacía del form (donde no haya otro control). Se retornará al editor de código y se habrá agregado otra función controlada por evento, llamada **Form1_Load**. Esta función se ejecutará cada vez que la ventana de ejecución del Form1 se haya presentado al usuario.
19. Para continuar, el código que usted debe insertar se ha dividido en bloques, diferenciados en la siguiente tabla.
 - Por cada bloque, ubique el cursor justo después de la llave de apertura del ámbito indicado y redacte ahí las instrucciones requeridas.
 - Si la función controladora de evento requerida aún no exista, proceda a crearla.

Objeto	y	Funcion controlada por Evento
	(clase Form1)	Form1_Load
•	Agregar el siguiente comentario: Evento se ejecuta al iniciar ejecución de Form1	
•	Asignar 0 a variable conta .	
•	Asignarle a mensa la siguiente cadena: "Aun no ha presionado botón Contar"	
•	Hacer que control label1 muestre el contenido de mensa , con la instrucción: <i>Label1.Text = mensa;</i>	
	btnContar	btnContar_Click
•	Digitar el siguiente código: <i>//Acumulador, Total veces que presiona botón</i> <i>conta = conta + 1;</i> <i>//conta+=1; //En forma de operador abreviado</i>	
•	Asignarle a mensa la siguiente cadena: <i>"Presiono botón Contar, un total de " + Convert.ToString(conta)+" veces";</i>	
•	Hacer que label1 muestre el contenido de mensa	
	btnReinicio	btnReinicio_Click
•	Digitar el siguiente código: <i>//Restaura conteo clic realizado a button1</i> <i>conta = 0;</i> <i>mensa = "Presiono boton Contar, un total de" + Convert.ToString(conta) + " veces";</i>	
•	Hacer que label1 muestre el contenido de mensa	
	BtnFin	BtnFin_Click
•	Digitar el siguiente código: <i>Close();//finaliza aplicación</i>	

20. Ejecute nuevamente la aplicación, observe el mensaje en la *label1*. De clic sobre el botón *Contar*. Observe que variable **conta** cuenta el total de clic hechos por usuario al control *btnContar*.
21. De clic varias veces en botón *Contar*. Verifique que se actualiza el conteo gestionado por la variable **conta**.
22. Presione el botón *Reiniciar conteo*. Verifique que el conteo comienza nuevamente en cero.
23. Vuelva a tocar el botón *Contar* varias veces y luego, haga clic en botón *Salir del programa*. El programa finaliza su ejecución y retorna a la etapa de diseño.

24. Guarde su proyecto, presionando opción *Archivo->Guardar Todo*.

PARTE 3: Tomas de decisiones (If Then Else)

Ahora procederá a solucionar este problema:

“Una administradora de préstamos requiere ingresar los datos de un préstamo (Monto a prestar, Tasa interés mensual y total de años (a 2 años inicialmente)) que se entregara a una Empresa ingresada. Determine el monto final que se pagara”.

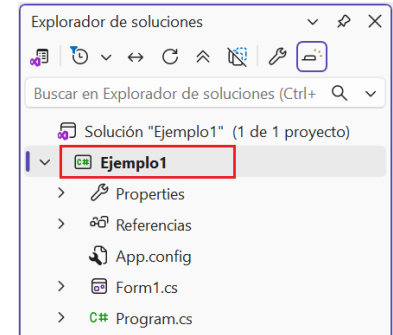
25. La solución del ejercicio de esta parte se hará con un nuevo formulario.

Para iniciar, localice la ventana del *Explorador de soluciones*.

Luego, en la jerarquía de elementos de su solución, haga clic secundario sobre el nombre de su proyecto (Ejemplo1) y seleccione la opción *Agregar -> Formulario (Windows Form..)*.

Se abre la ventana *Agregar nuevo elemento*. En la parte inferior, cambie el nombre ofrecido (Form2) por **frmPrestamo**.

Observe que se agrega el form *frmPrestamo* a su proyecto y se muestra su diseño en una ficha.



26. Para que su aplicación inicie con el nuevo form, retorne a la ventana del *Explorador de soluciones* y haga doble clic sobre el archivo **Program.cs**, para ver su contenido en otra ficha.

27. Dentro del contenido, localice la llamada al método Run de la clase Application. Cambie el nombre del Form1 por el nuevo form, así:

```
Application.Run(new frmPrestamo());
```

28. Guarde los cambios del proyecto actual y nuevamente ejecute su aplicación.

29. Cierre la vista de ejecución del nuevo form para retornar al diseño.

30. Abra la ficha de diseño del frmPrestamo y luego, ubica a los controles listados en la imagen 8.

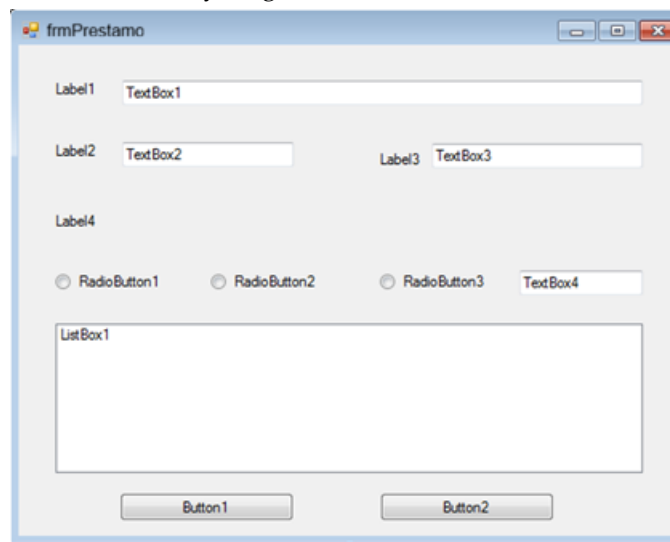


Imagen 8: Diseño de controles del form frmPrestamo.

31. Con mucho cuidado, seleccione a cada control mostrado en la tabla a continuación, para luego modificar a c/propiedad solicitada ahí con el valor indicado.

(propiedades)				
(controles)	Name	Text	Enabled	Checked
frmPrestamo		Calculo del Monto a pagar por un Prestamo		
Label1		Nombre de Empresa:		
Label2		Monto prestamo (\$)		
Label3		Tiempo (años)		
Label4		Tasa Interes:		
RadioButton1	rdbInteres1	12 %		true
RadioButton2	rdbInteres2	23.5 %		
RadioButton3	rdbInteres3	Otro, indique:		
TextBox1	txtEmpresa	(vacío)		
TextBox2	txtMonto	(vacío)		
TextBox3	txtTiempo	2		
TextBox4	txtTasaInterEX	0	false	
Button1	btnAnalisis	Análisis Financiero		
Button2	btnFin	Salir del programa		
ListBox1	lstResul		false	

* **Extra:** Para el cuadro *lstResul*, seleccione además a su propiedad **Items**, y presione en *Colecciones (...)*. Luego, en lista vacía, digitar valor: **(Resultados)**. Esto agregará este texto como 1er elemento del control *lstResul*

32. Presione la tecla F7 para cambiar a la vista de edición de la clase del form *frmPrestamos*. Realice la siguiente asignación de código, de acuerdo con el ámbito del objeto y del evento indicados a continuación.

Objeto y Evento/Nivel	
frmPrestamos	(a nivel de Clase)
- Declarar variable: TasaI de tipo double. - Digitar el siguiente código para una nueva función llamada IsNumeric: <pre>//valida que el dato recibido es un numero Boolean IsNumeric(string valor) { int result; return int.TryParse(valor, out result); }</pre>	
rdbInteres3	rdbInteres3_CheckedChanged
Digitar el siguiente código: <pre>if (rdbInteres3.Checked == true) { txtTasaInterEX.Enabled = true; txtTasaInterEX.Focus(); } else { txtTasaInterEX.Text = "0"; }</pre>	

txtTasaInterEX.Enabled = false;	
}	
rdbInteres1	Click
• Asignar a TasaI el valor de 0.12	
rdbInteres2	Click
• Asignar a TasaI el valor de 0.235	
btnFin	Click
• Invocar a la función Close para finalizar la ejecución de su aplicación.	
btnAnalisis	Click
<pre> //Declaracion de variables a utilizar string NomEmpre; double MontoInic=0, MontoFin=0; int Tiempo; NomEmpre = txtEmpresa.Text; NomEmpre = NomEmpre.Trim();//Quita el espacio-blanco al inicio-final if (NomEmpre.Length == 0) { //si no hay caracteres en nombre empresa MessageBox.Show("Debe indicar Nombre de la empresa", "ERROR", MessageBoxButtons.OK, MessageBoxIcon.Information); txtMonto.Focus(); //metodo que indica que control txtEmpresa recibira cursor return;//sale del procedimiento btnanalisis } if (!(IsNumeric(txtMonto.Text))) { MessageBox.Show("Valor Monto incorrecto", "ERROR", MessageBoxButtons.OK, MessageBoxIcon.Error); txtMonto.Focus(); //metodo que indica que control txtEmpresa recibira cursor return; } else { MontoInic = Convert.ToDouble(txtMonto.Text); if (!(MontoInic > 0)){ MessageBox.Show("Valor Monto no puede ser negativo", "ERROR", MessageBoxButtons.OK, MessageBoxIcon.Error); txtMonto.Focus(); //metodo que indica que control txtEmpresa recibira cursor return; } } Tiempo = Convert.ToInt32(txtTiempo.Text); //si selcciono Tasa interes 3, valida que sea correcta txtTasaInterEX.Text = txtTasaInterEX.Text.Trim(); if (rdbInteres3.Checked == true) { if (txtTasaInterEX.Text.Length > 0){ if (!(IsNumeric(txtTasaInterEX.Text) == true)) { MessageBox.Show("Tasa interes incorrecto", "ERROR", MessageBoxButtons.OK, MessageBoxIcon.Error); txtTasaInterEX.Text = "0"; } } } </pre>	

```

        txtTasaInterEX.Focus();
        return;
    }
    else {
        TasaI = Convert.ToDouble(txtTasaInterEX.Text) / 100;
    }
}
else
{
    MessageBox.Show("Aun no ha indicado una tasa interes", "ERROR",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
    txtTasaInterEX.Focus();
    return;
}
}

//Hace el cálculo esperado
MontoFin =(1+TasaI);
MontoFin = MontoInic*(Math.Pow(Convert.ToDouble(MontoFin), Tiempo));
TasaI *= 100;
//Muestra la respuesta (Monto a pagar)
lstResul.Items.Clear();
lstResul.Items.Add("Empresa: " + txtEmpresa.Text);
lstResul.Items.Add("Monto: $" + MontoInic + ", Tasa anual: " + TasaI+ " %");
lstResul.Items.Add("Monto a pagar: $" + MontoFin);

```

33. Guarde los cambios del proyecto actual y ejecute la aplicación.

Para ver las validaciones de entradas programadas, equivóquese a propósito en el ingreso de valores numéricos a ingresar y presione boton *btnAnalisis*

34. Presione el botón *Salir del programa*, para así retornar al modo de diseño.

PARTE 4: Tomas de decisiones (Switch)

Problema a solucionar:

Solicite cada apellido y cada nombre de un estudiante, así como el valor del CUM obtenido de su último ciclo de estudios.

Determine el total de unidades valorativas que un estudiante puede llevar en el próximo ciclo de estudios según el valor del CUM ingresado. Cuando el CUM es mayor a 7.5, puede llevar 32 UV máximo, si tiene entre 7 hasta 7.5, llevaría 24 UV Max, si tiene CUM entre 6 a 6.9, cursaría 20 UV.

Y finalmente, con un CUM de 1 a 5.9, podrá llevar 16 unidades valorativas. En cualquier otro caso cursara 0 UV.

35. Ubique nuevamente a la ventana del *Explorador de soluciones* y agregue un nuevo formulario a su proyecto, con el nombre de **frmCUM**.
36. Abra el archivo *Program.cs* de su proyecto y modifique la instancia del formulario a iniciar por el nuevo form **frmCUM**, así:

```
Application.Run(new frmCUM( ));
```

37. Luego, en este nuevo form (**frmCUM**) diseñe el esquema de objetos mostrado en la imagen 9.

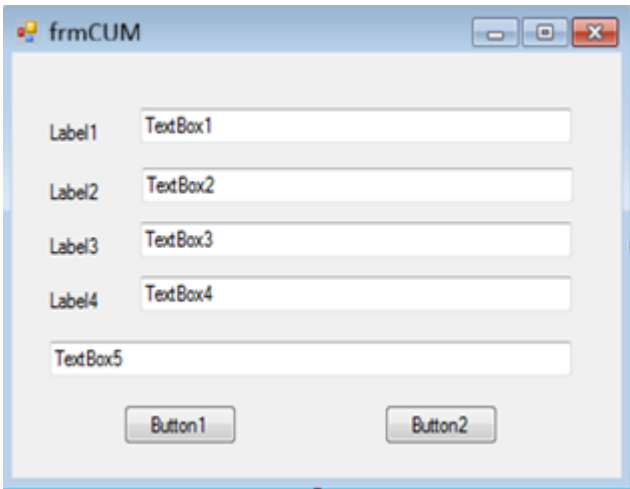


Imagen 9: Diseño de controles del frmCUM

(controles)	(Propiedades)		
	Name	Text	Enabled
frmCUM		Calculo de UV a cursar	
Label1		1er Apellido:	
Label2		2do Apellido:	
Label3		Nombres	
Label4		Valor de CUM	
TextBox1	txtApe1	(vacio)	
TextBox2	txtApe2	(vacio)	
TextBox3	txtNom	(vacio)	
TextBox4	txtCUM	0.0	
TextBox5	txtResul	Resultado de evaluación	false
Button1	btnAnalisis	Calcular UV	
Button2	btnFin	Salir del programa	

38. Proceda a digitar el código detallado a continuación, en cada evento y objeto especificado.

Objeto y Ambito y/o Evento del código a incluir
(a nivel de clase frmCUM)
- Declarar variables noms, ape1 y ape2 de tipo string. - Declarar variable CUM de tipo double. - Declarar variable UV de tipo int. - Digitar el siguiente codigo: <pre style="margin-left: 20px;"> //valida que el dato recibido es un numero public Boolean IsNumeric(string valor) { int result; return int.TryParse(valor, out result); } //evalua el cum private void EvaluarCUM() { /* una vez recibidos los nombres del estudiante, asi como su CUM se detemina las UV' solicitadas */ string nombrecompleto; nombrecompleto = noms + " " + ape1 + " " + ape2; nombrecompleto = nombrecompleto.ToUpper() ; </pre>

```

if (CUM < 0 | CUM > 10)
{
    MessageBox.Show("Valor de CUM fuera de rango (0.0 - 10.0)",
        "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    txtCUM.Focus();
}
else {
    //Usa estructura switch
    switch (Convert.ToInt16(CUM))
    {
        case 8 - 10:
            UV = 32;
            break;
        case 7:
            UV = 24;
            break;
        case 6:
            UV = 20;
            break;
        case 1: case 2: case 3:
        case 4: case 5:
            UV = 16;
            break;
        default:
            UV = 0;
            break;
    }
    txtResul.Text = nombrecompleto + "Puede cursar " + UV + "UV";
}
}

```

btnAnalisis	Click
- Realizar las verificaciones que crea necesarias para determinar si los valores dados por usuario son los apropiados. Aquí también debe usar la función <i>IsNumeric</i>. - Llamar a <i>EvaluarCUM</i>, donde usted crea conveniente.	
btnFin	Click
Instrucción para finalizar aplicación.	

39. Guarde los cambios del proyecto actual y ejecute la aplicación.

Para ver las validaciones de entradas programadas, equívóquese a propósito, ingrese el CUM, pero olvídense de los nombres, y otros casos.

40. En evento Click del botón *btnFin* redacte un mensaje de despedida y realice el final de ejecución

V. ANALISIS DE RESULTADOS**PROBLEMAS PARA RESOLVER:**

Dentro de la carpeta general creada al inicio del procedimiento, cree un proyecto, con la plantilla Windows Forms (.net framework). En el interior de este proyecto, desarrollar la solución de cada uno de los problemas descritos a continuación. Cada solución debe elaborarse en formularios distintos.

1. Solicitar 10 números decimales al usuario, para luego mostrar solamente a uno de estos resultados (según lo seleccione usuario) del conjunto de números ingresados previamente:

- El número mayor de solamente los negativos del listado ingresado hasta ese momento
- Valor del promedio de solamente los números positivos.
- Mostrar el valor de la amplitud (o rango) de toda la serie de los números ingresados.

2. Ayude a un usuario a ingresar una fecha de manera correcta, en el formato: Mes, Día, Año.

Para el diseño, debe utilizar controles Combobox, y estructuras switch en su código.

Recuerde que existen años bisiestos.

3. Permita el ingreso de 10 números decimales cuales quiera. Cada nuevo número ingresado se mostrará en una de 2 listas diferentes (una solamente para valores positivos (y ceros) y en la segunda, solo para los valores negativos).

Una vez finalice el ingreso, se mostrará en otro listado a la solución a estas incógnitas:

- ¿Cuál fue el menor de los números negativos?
- ¿Promedio de los números positivos?

Importante:

+ En todos los ejercicios, documente los bloques y líneas de código de manera apropiada, para que su instructor comprenda mejor su solución.

+ Finalmente, comprima la carpeta general elaborada para esta Guía 1, conteniendo ambos proyectos (del procedimiento y del análisis) y envíe el archivo resultante hacia donde y en la fecha que su instructor lo indique.

VII. BIBLIOGRAFÍA

• Título: Aprenda ya Microsoft Visual C#.NET.

Autores: Sharp, John Jagger, Jon Coautor

Publisher: Madrid, España: McGraw-Hill, 2002.

Clasificación: Libro 005.362 S581 2002

• Título: C # Manual de Programación.

Autores: Luis Joyanes Aguilar y Matilde Fernández Azuela

Publisher: Madrid, España: McGraw-Hill, 2002

Clasificación: Libro 005.362 J88 2002